

Dr. Mahmoud Alnamoly

x

07

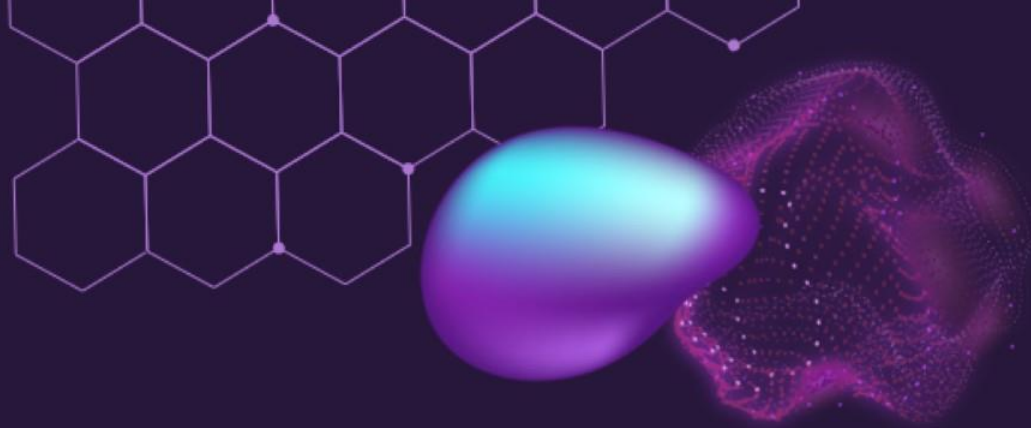
Tree

Dr. Mahmoud Alnamoly

x

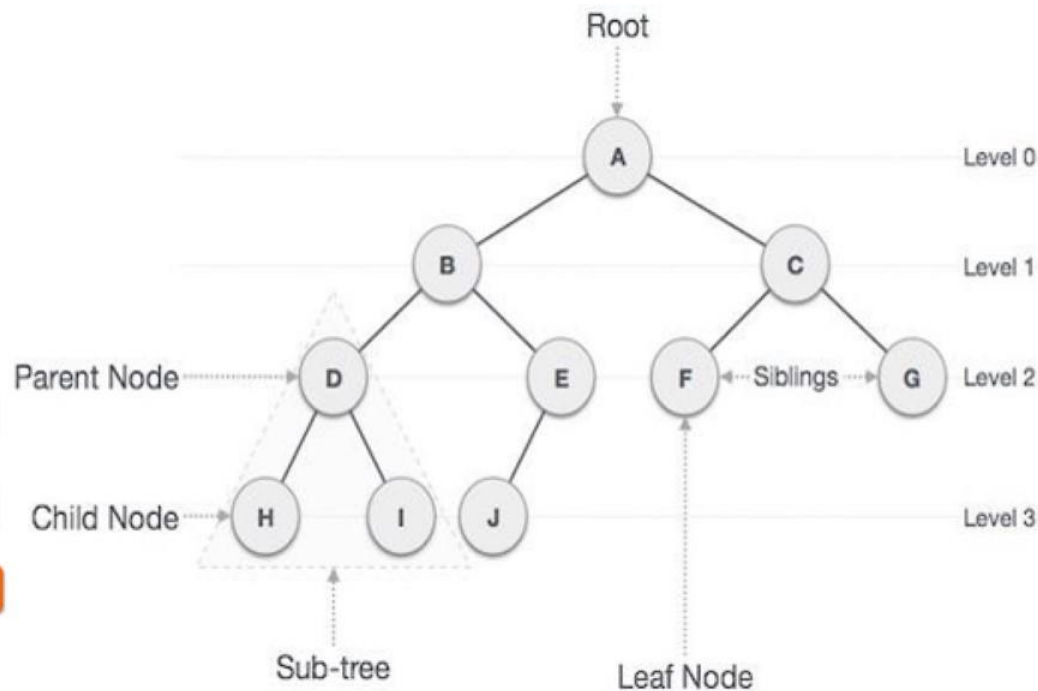
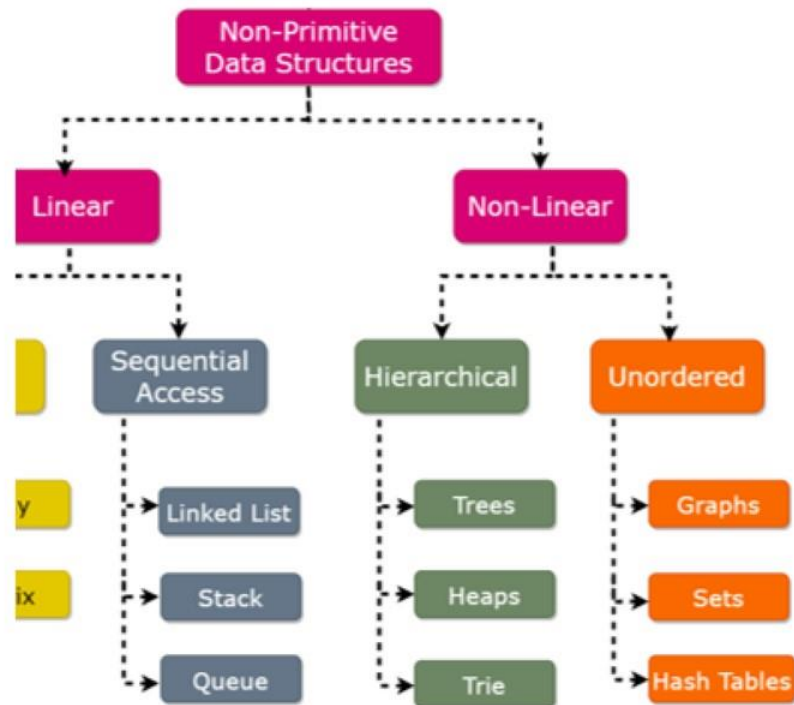
+

Dr. Mahmoud Alnamoly



Tree ?

A tree is a **nonlinear** hierarchical data structure that consists of nodes connected by edges.



Types of Tree Data Structure

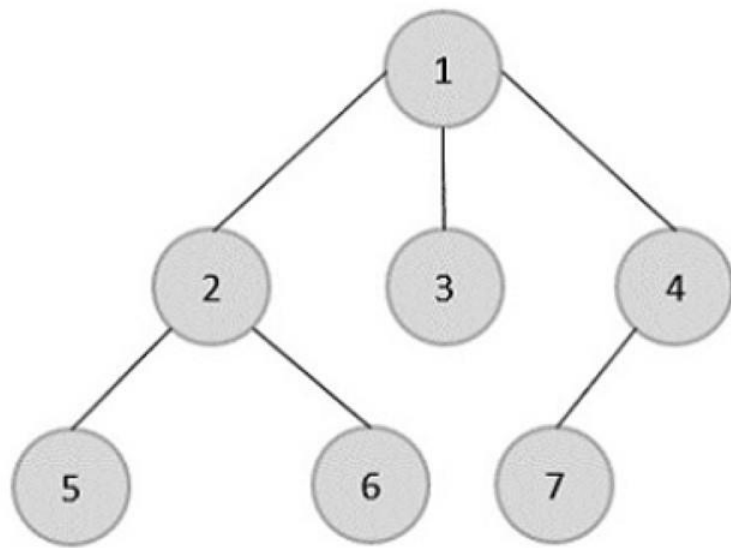
- ✓ General Tree
- ✓ Binary Tree
- ✓ Binary Search Tree (BST)
- ✓ AVL Tree
- ✓ Red-Black Tree
- ✓ Heap
- ✓ Prefix Tree
- ✓ B-Tree
- ✓ Segment Tree
- ✓ Suffix Tree
- ✓

Dr. Mahmoud Alnamoly

General Tree

General trees are tree data structures where the root node has **minimum 0 or maximum 'n'** subtrees.

The General trees have no constraint placed on their hierarchy.



General trees which have 3 sub-trees per node are called **ternary trees**.

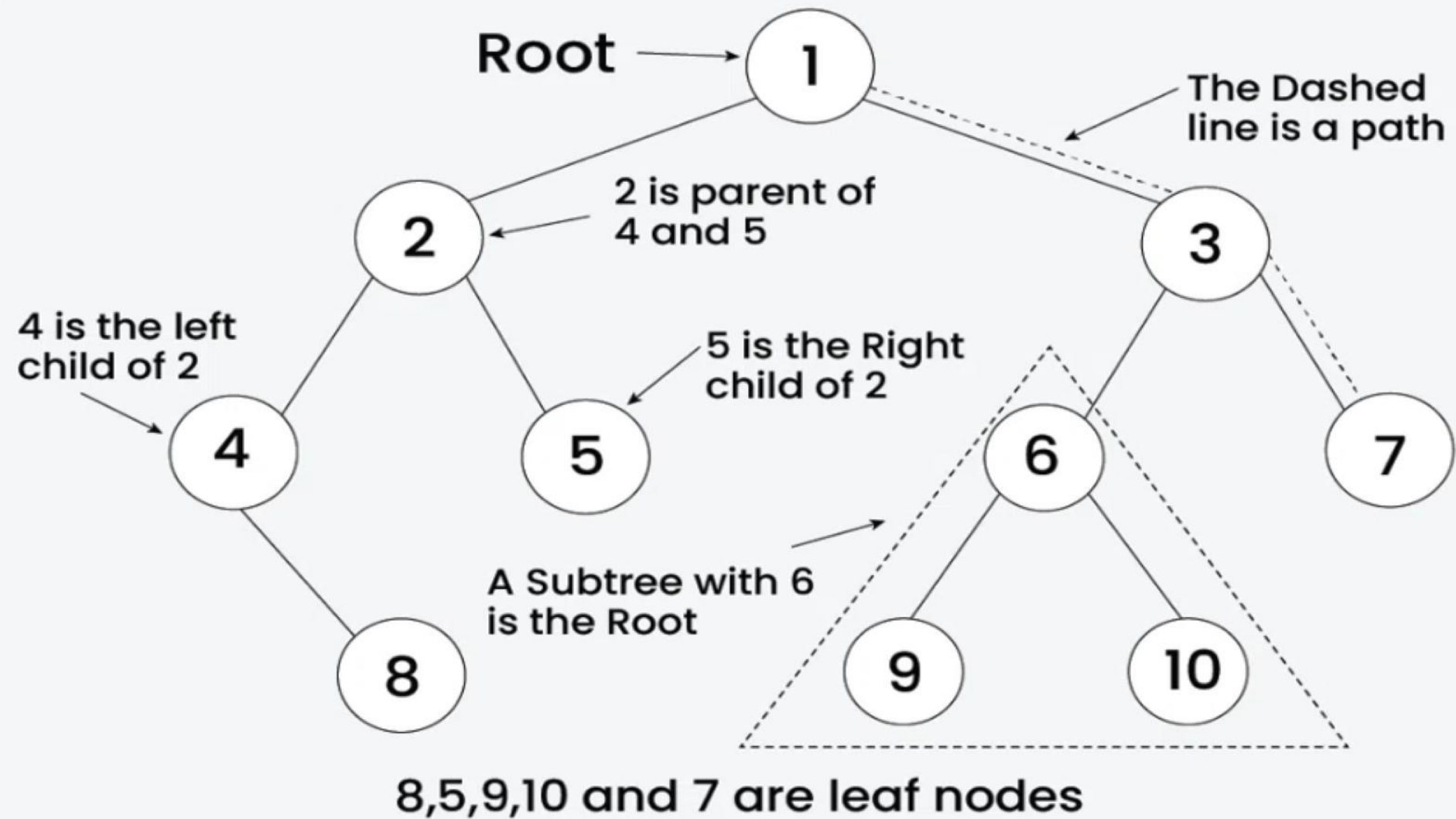
Binary Tree

Binary Tree: Definition

Dr. Mahmoud Alnamoly

Binary Tree is a non-linear and hierarchical data structure (general tree), where each node has at most two children referred to as the left child and the right child.

The topmost node in a binary tree is called the root, and the bottom-most nodes are called leaves.



Binary Tree: Variations

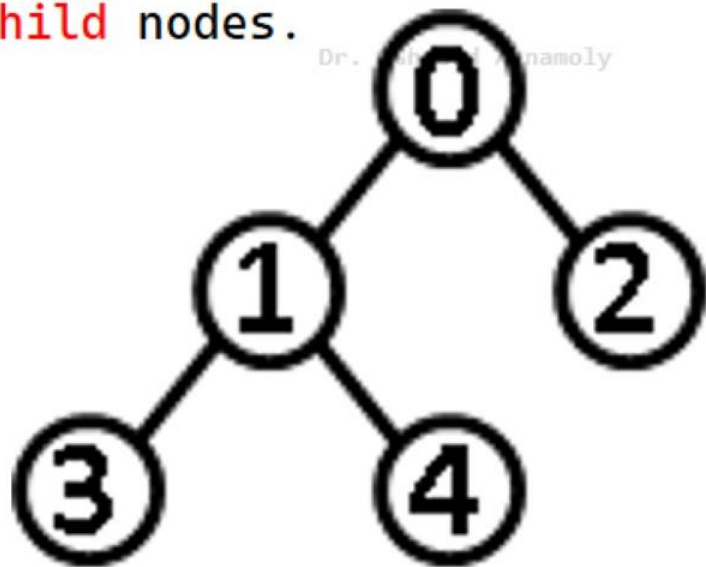
Dr. Mahmoud Alnamoly

Based on the number of children, binary trees are divided into **three types**:

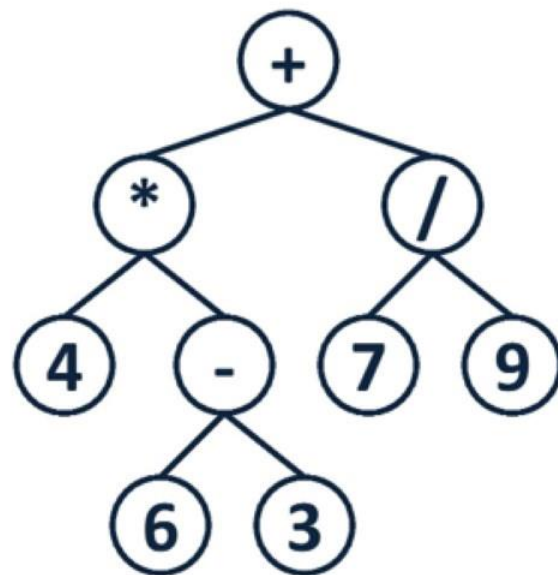
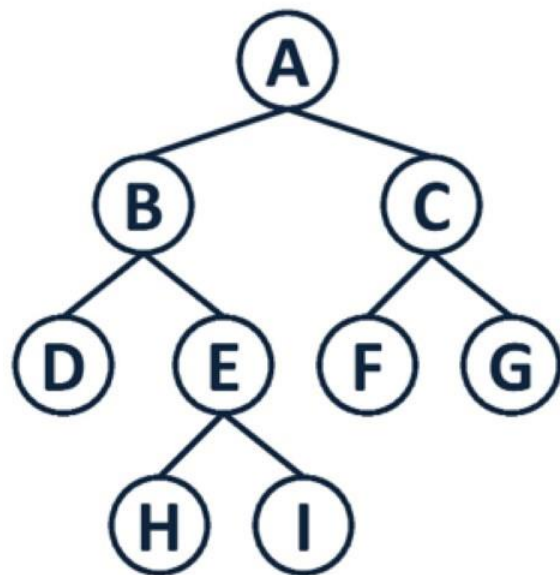
Full Binary Tree

A full binary tree is a binary tree type where every node has either **0 or 2** child nodes.

Dr. Mahmoud Alnamoly



Full Binary Tree



Expression Tree for
 $(4*(6-3))+(7/9)$

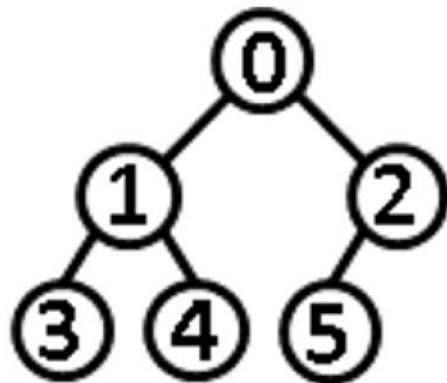
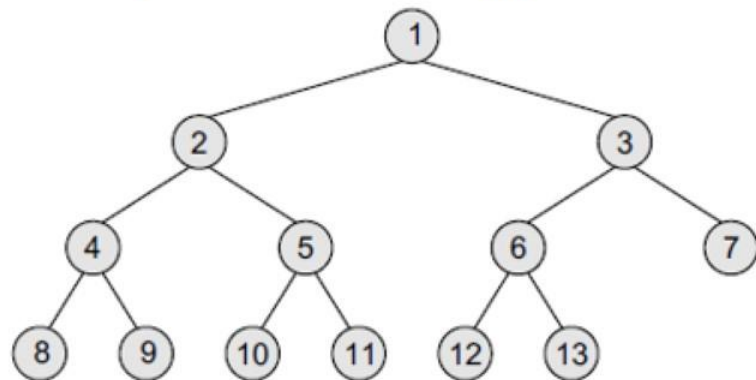
Binary Tree: Variations

Dr. Mahmoud Alnamoly

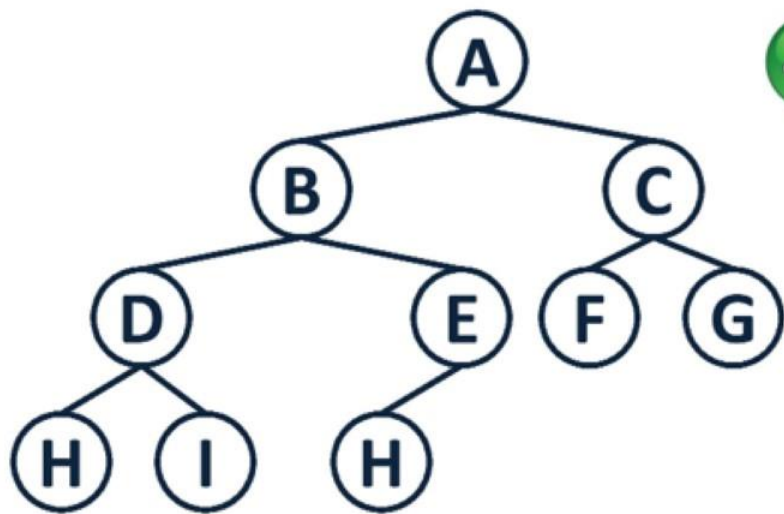
Based on the number of children, binary trees are divided into **three types**:

Complete binary tree is a binary tree that satisfies two properties.

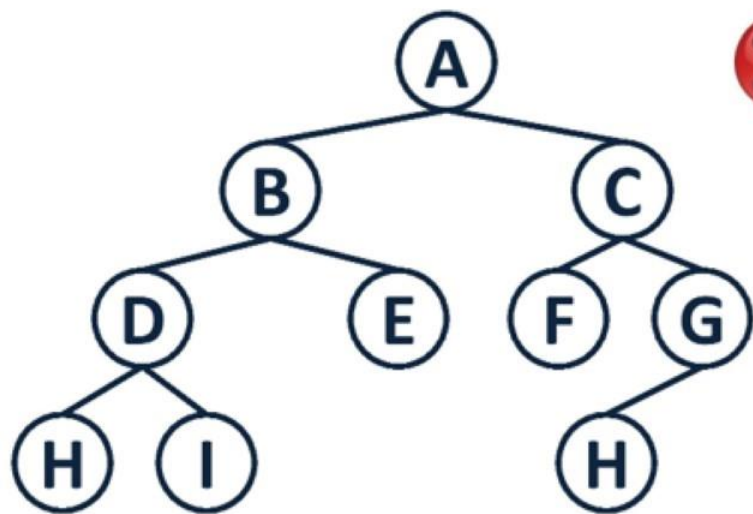
- First, every level, except possibly the last, is completely filled. However, root and internal nodes in a complete binary tree can either have 0, 1 or 2 child nodes.
- Second, all nodes appear as far left as possible.



Complete Binary Tree



It's a complete Binary Tree, as all the nodes except last level has two children.



It's not a complete Binary Tree, as E does not have any children, but G has.

Binary Tree: Variations

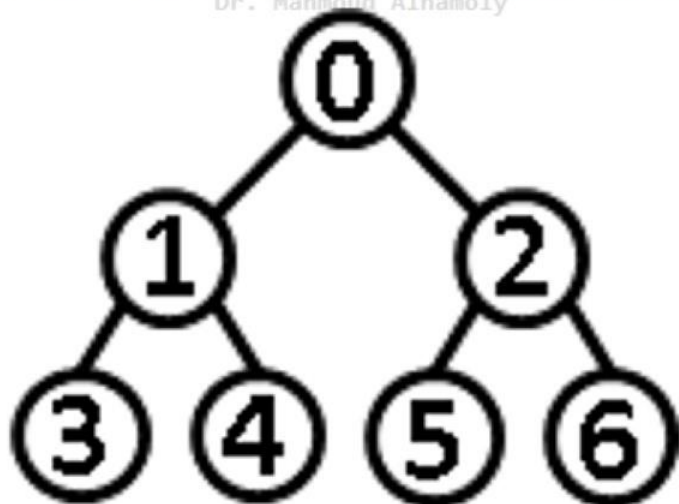
Dr. Mahmoud Alnamoly

Based on the number of children, binary trees are divided into **three types**:

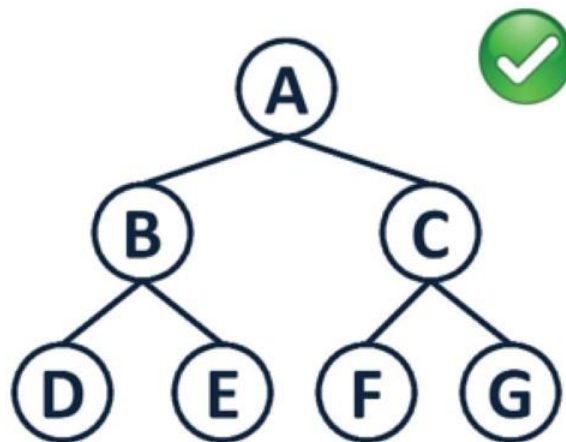
Perfect Binary Tree

A perfect binary tree is a binary tree type where all the leaf nodes are on the same level and every node except leaf nodes have 2 children.

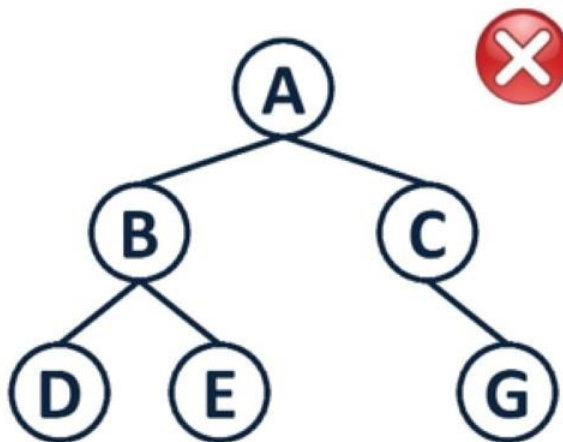
Dr. Mahmoud Alnamoly



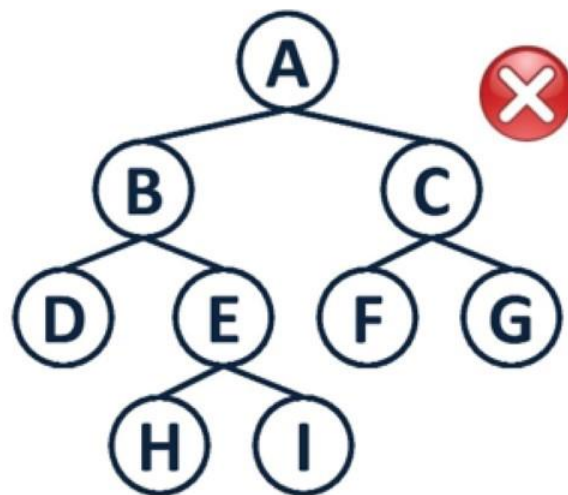
Perfect Binary Tree



This tree is a Perfect Binary Tree as all internal nodes has exactly two children and all leaf nodes are on the same level



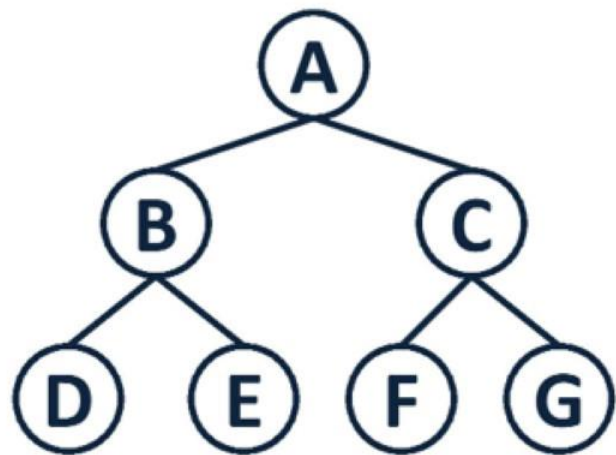
Above tree is not a perfect Binary Tree as node C has only one child.



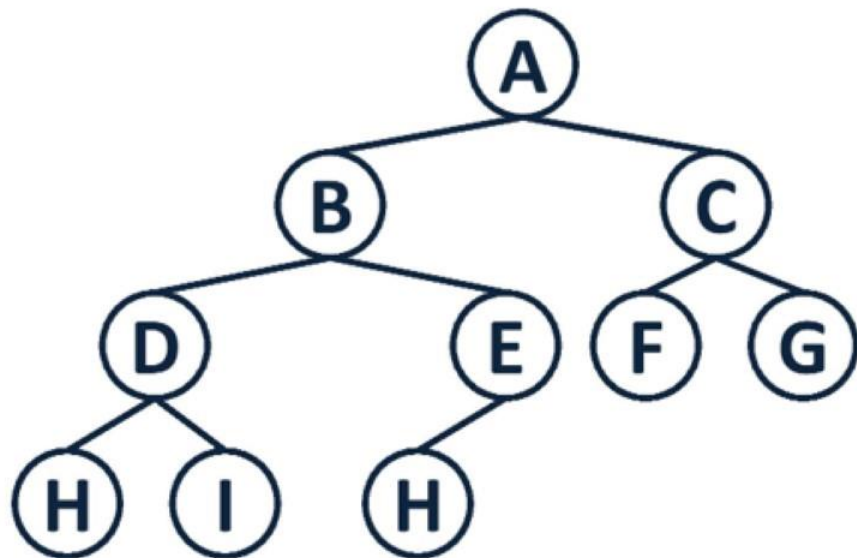
Above tree is not a perfect Binary Tree as all leaf nodes are not on the same level

Note:- A Perfect Binary Tree whose rightmost leaves (perhaps all) on the last level have been removed is called **Complete**

Perfect vs Complete Binary Tree



All Perfect Binary Trees are Complete Binary Trees, but reverse is not true.



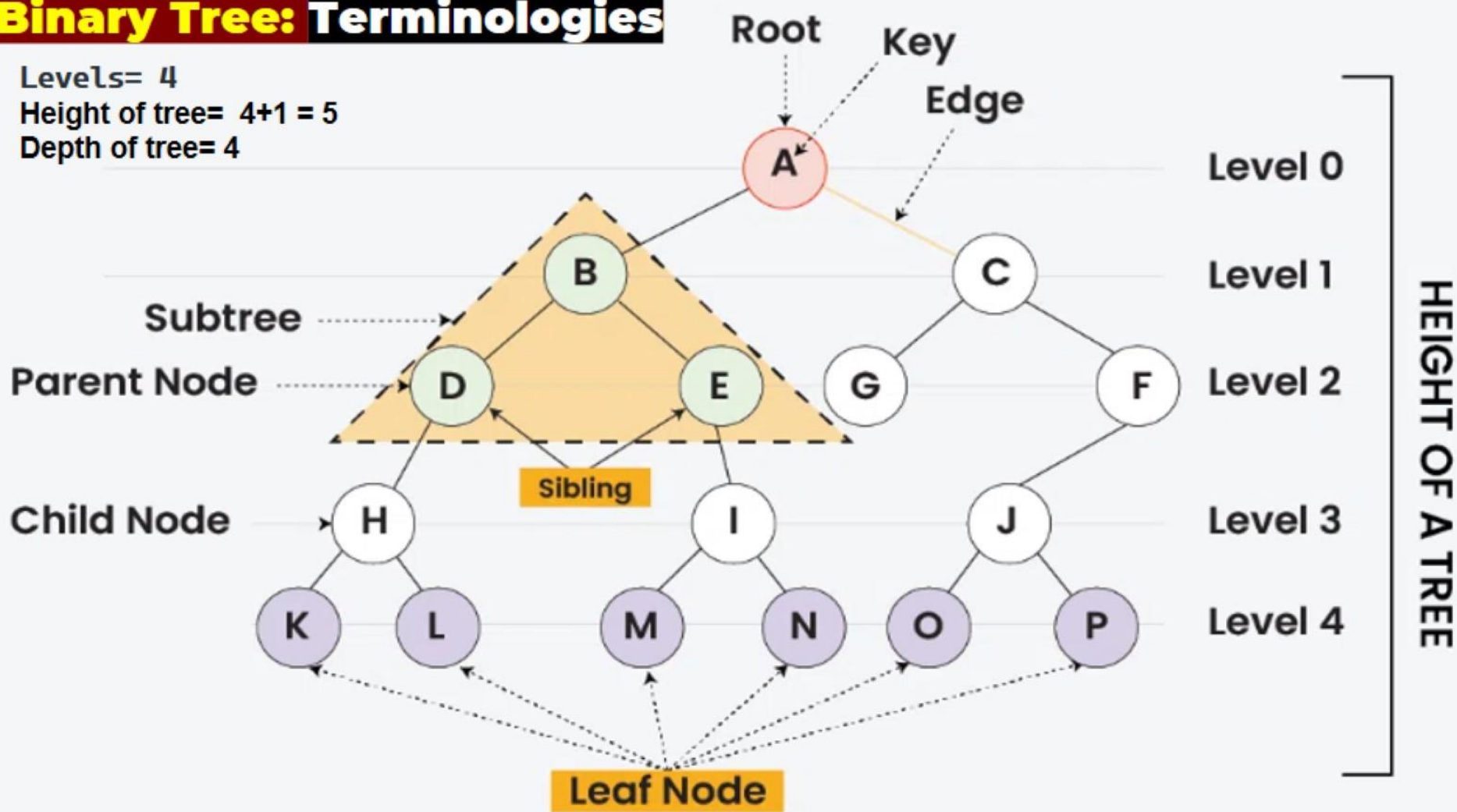
It is Complete Binary Tree, but not a Perfect Binary Tree.

Binary Tree: Terminologies

Levels= 4

Height of tree= $4+1 = 5$

Depth of tree= 4



Binary Tree: Terminologies

- ✓ **Nodes**: The fundamental part of a binary tree, where each node contains data and link to two child nodes.
- ✓ **Root**: The topmost node in a tree is known as the root node. It has no parent and serves as the starting point for all nodes in the tree.
- ✓ **Parent Node**: A node that has one or more child nodes. In a binary tree, each node can have at most two children.
- ✓ **Child Node**: A node that is a **descendant** of another node (its parent).
- ✓ **External nodes (terminal nodes- Leaf Node)**: A node that does not have any children or both children are null.
- ✓ **Internal Node**: A node that has at least one child. This includes all nodes except the **root** and the **leaf** nodes.

Binary Tree: Terminologies

- ✓ **Edge:** A connection between two nodes. It represents the relationship between a parent and a child node.
- ✓ **Path:** A sequence of consecutive edges.
- ✓ **Subtree:** The Subtree is a portion of the tree that includes a node and all its descendants
- ✓ **Level:** The level of a node is defined by its depth. The root node is at level 0, its children are at level 1, and so on. The maximum number of nodes at any level L is 2^L .
- ✓ **Depth of a Node:** The number of edges (length of the path) from a specific node (N) to the root node (R). The depth of the root node is zero. The maximum number of nodes in a binary tree of depth d is $(2^d)-1$, where $d \geq 1$.
- ✓ **Height of a Binary Tree:** The total number of nodes on the path from the root node to the deepest leaf node in the tree. A tree with only a root node has a height of 1.

Binary Tree: Terminologies

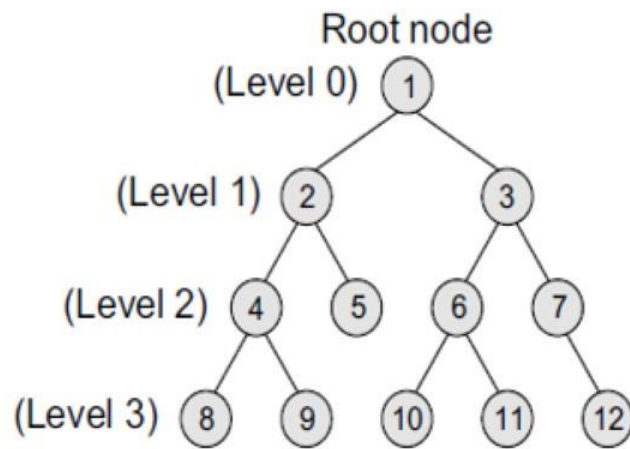
Dr. Mahmoud Alnamoly

Degree of a node:-

It is equal to the number of children that a node has.

The **degree** of a leaf node is zero.

For example, in the tree, degree of node 4 is 2, degree of node 5 is zero and degree of node 7 is 1.



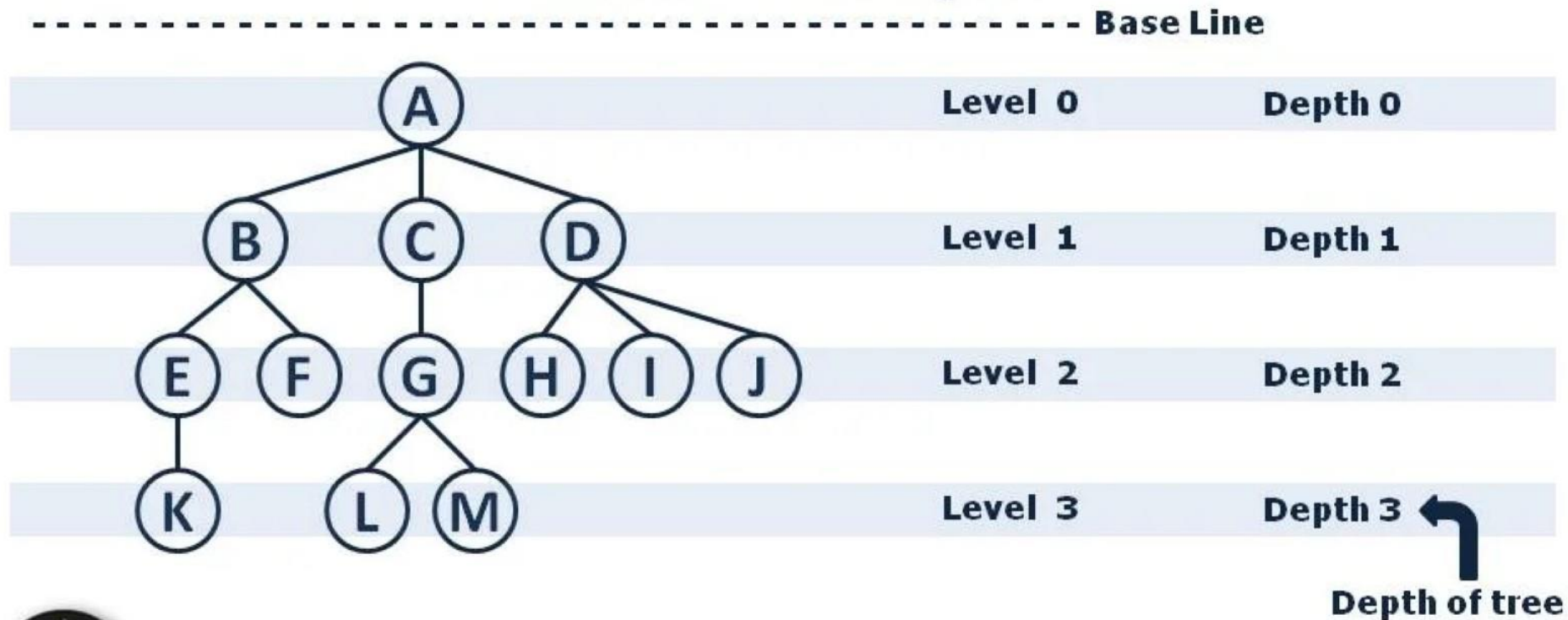
In-degree/out-degree of a node:-

In-degree is the number of edges arriving at a node.

The root node is the only node that has an in-degree equal to zero.

Similarly, **out-degree** of a node is the number of edges leaving that node.

Level and Depth



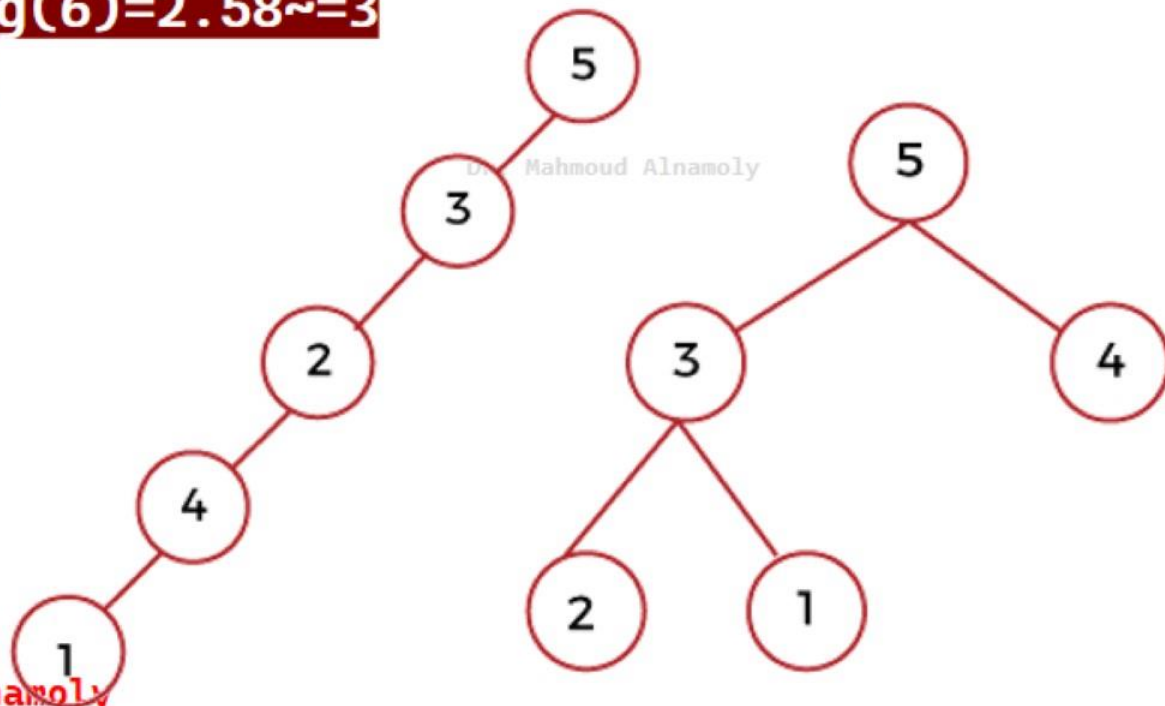
The height of binary tree with N nodes is at least $\log_2(N+1)$ and at most N

Example:-

Nodes=5

Min_H = $\log(6) = 2.58 \sim 3$

Max_H = N = 5



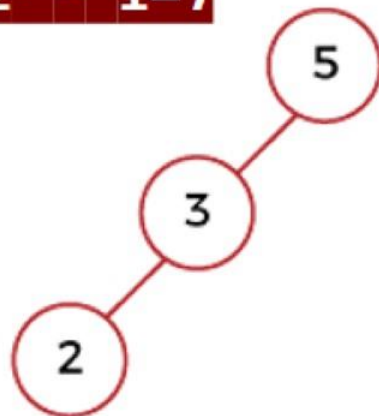
The number of nodes on binary tree with height H is at least H and at most $2^H - 1$

Example:-

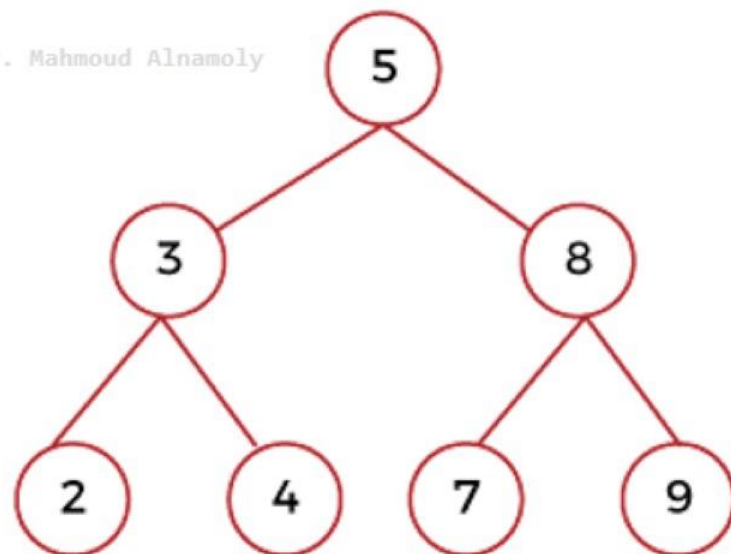
Height=3

Min_Nodes= $H = 3$

Max_Nodes= $2^H - 1 = 7$



Dr. Mahmoud Alnamoly



Binary Tree: Representation

Dr. Mahmoud Alnamoly

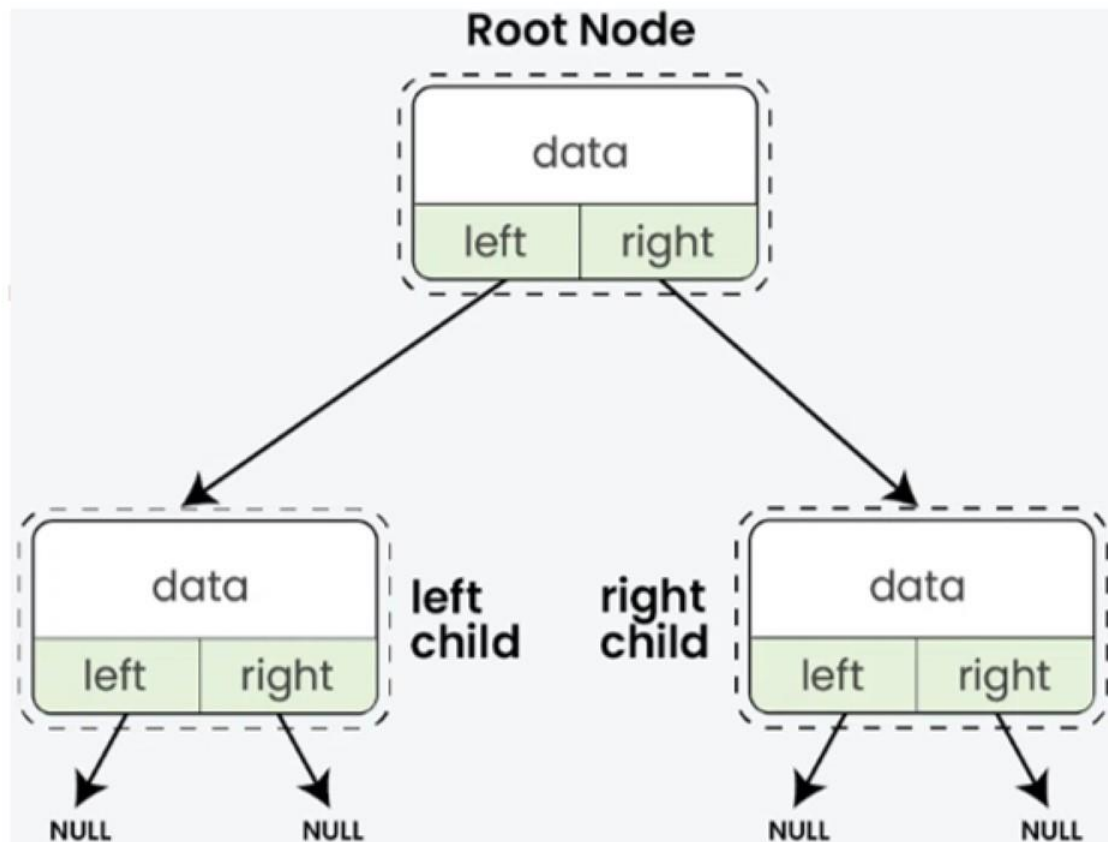
Each node in a Binary Tree has three parts:

- Data
- Pointer to the left child
- Pointer to the right child

// Node Structure

```
struct Node {  
    int data;  
    Node* left;  
    Node* right;  
  
    Node(int value) {  
        data = value;  
        left = right = nullptr;  
    }  
};
```

Dr. Mahmoud Alnamoly



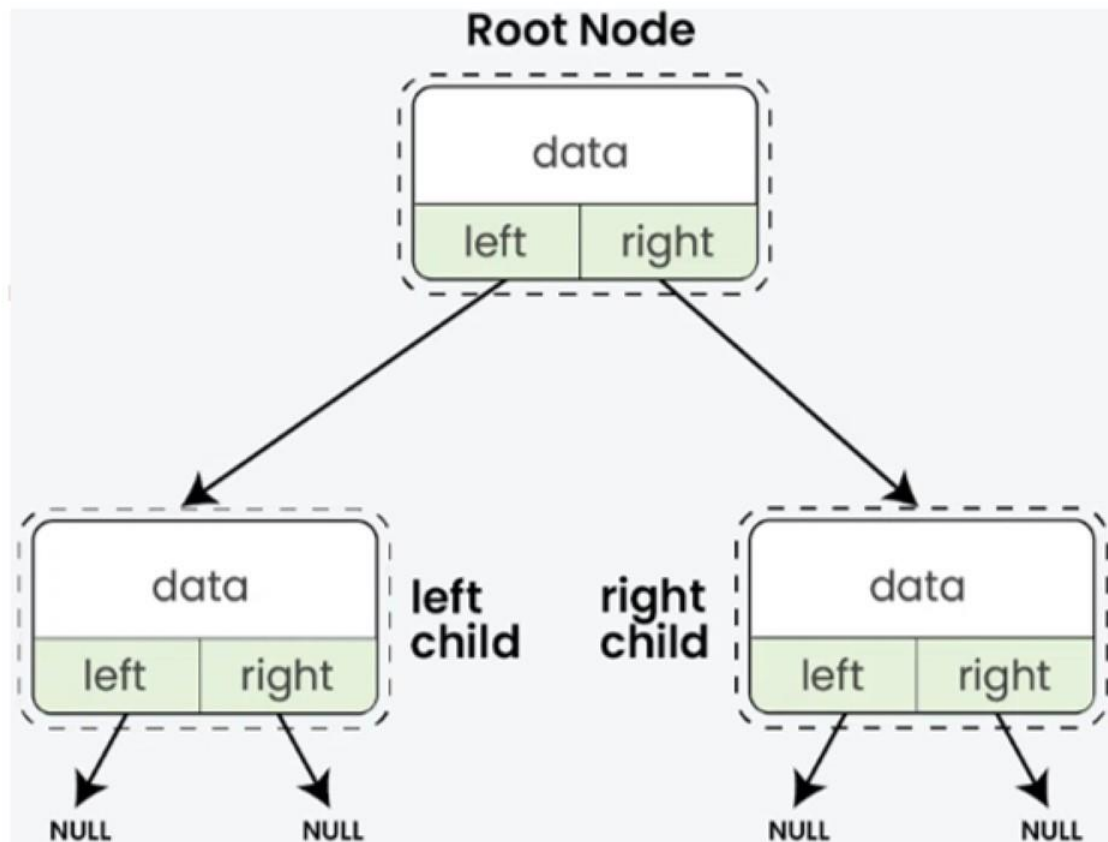
Binary Tree: Representation

Dr. Mahmoud Alnamoly

Each node in a Binary Tree has three parts:

- Data
- Pointer to the left child
- Pointer to the right child

```
// BinaryTree class
class BinaryTree {
private:
    Node* root;
public:
    // Constructor
    BinaryTree() {
        root = nullptr;
    }
}
```



Binary Tree: Operations

Dr. Mahmoud Alnamoly

✓ 1. Traverse

Traversal in Binary Tree involves visiting all the nodes of the binary tree.

Tree Traversal algorithms can be classified broadly into two categories, **BFS** and **DFS**:

Breadth-First Search (BFS) algorithms:

BFS explores all nodes at the present depth before moving on to nodes at the next depth level.

It is typically implemented using a **queue**.

BFS in a binary tree is commonly referred to as Level Order Traversal.

Binary Tree: Operations

Dr. Mahmoud Alnamoly

✓ 1. Traverse

Traversal in Binary Tree involves visiting all the nodes of the binary tree.

Tree Traversal algorithms can be classified broadly into two categories, **DFS** and **BFS**:

Depth-First Search (DFS) algorithms: DFS explores as far down a branch as possible before backtracking. It is implemented using **recursion**.

The main traversal methods in DFS for binary trees are:
Preorder-Inorder-Postorder

✓ Traversal: Depth First Search Algorithm (Pre-order)

// Helper function for preorder traversal

```
void preorder(Node* node) {  
    if (node == nullptr) {  
        return;  
    }  
  
    cout << node->data << " ";  
    preorder(node->left);  
    preorder(node->right);  
}
```

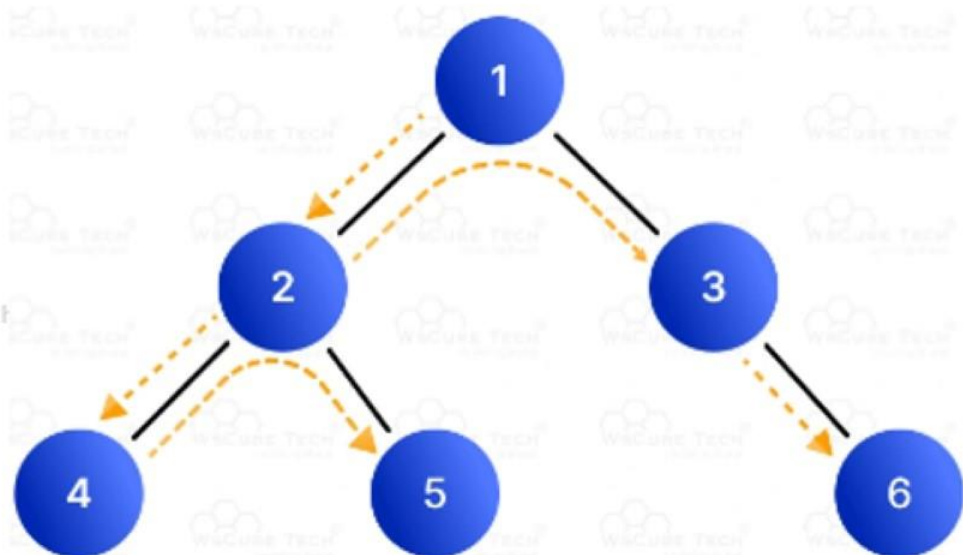
// Preorder traversal of the tree

```
void preorder() {  
    preorder(root);  
    cout << endl;  
}
```

BinaryTree tree;

```
cout << "Preorder traversal: ";  
tree.preorder();
```

Dr. Manish Arora



Preorder Traversal: 1→2→4→5→3→6

current-left-right

Visits the node first, then left subtree, then right subtree

✓ Traversal: Depth First Search Algorithm (In-order)

// Helper function for inorder traversal

```
void inorder(Node* node) {  
    if (node == nullptr) {  
        return;  
    }  
  
    inorder(node->left);  
    cout << node->data << " ";  
    inorder(node->right);  
}
```

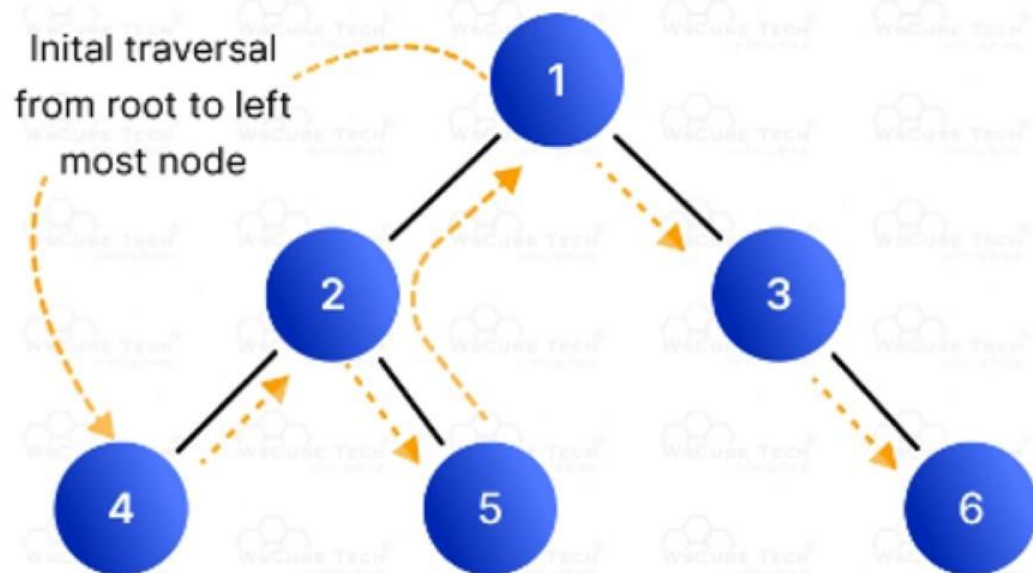
// Inorder traversal of the tree

```
void inorder() {  
    inorder(root);  
    cout << endl;  
}
```

BinaryTree tree;

cout << "Inorder traversal: ";

tree.inorder();



Inorder Traversal: 4→2→5→1→3→6

Left-current-right

Visits left subtree, then the node, then the right subtree.

✓ Traversal: Depth First Search Algorithm (Post-order)

// Helper function for postorder traversal

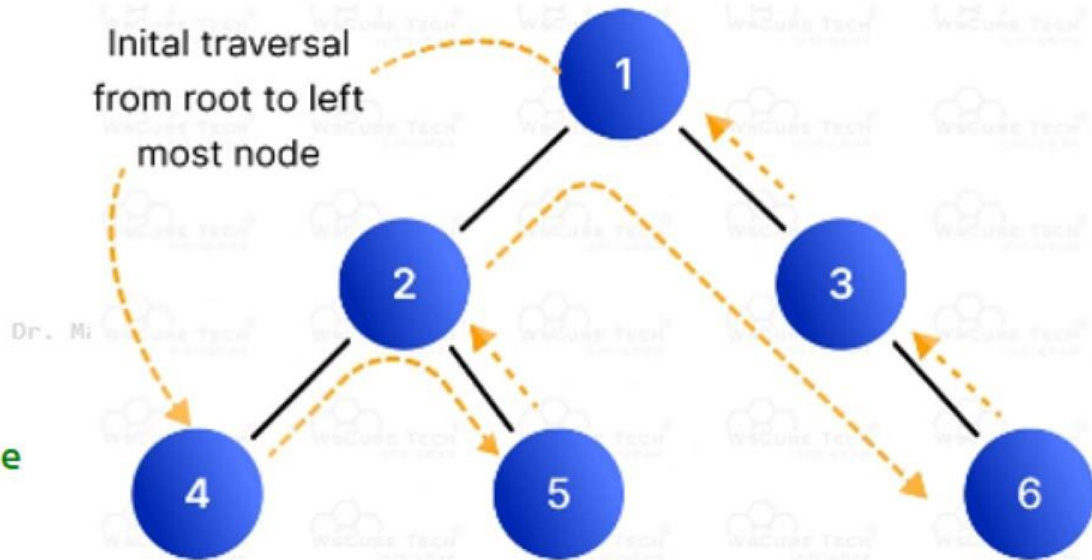
```
void postorder(Node* node) {  
    if (node == nullptr) {  
        return;  
    }  
  
    postorder(node->left);  
    postorder(node->right);  
    cout << node->data << " ";  
}
```

// Postorder traversal of the tree

```
void postorder() {  
    postorder(root);  
    cout << endl;  
}
```

BinaryTree tree;

```
cout << "Postorder traversal: ";  
tree.postorder();
```



Postorder Traversal: 4→5→2→6→3→1

left-right-current

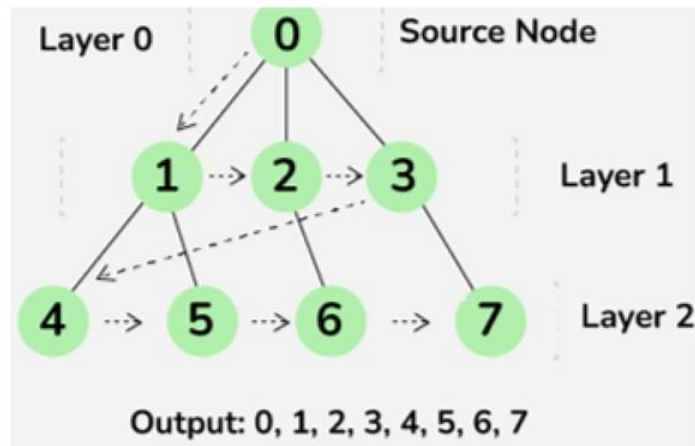
Visits left subtree, then right subtree, then the node.

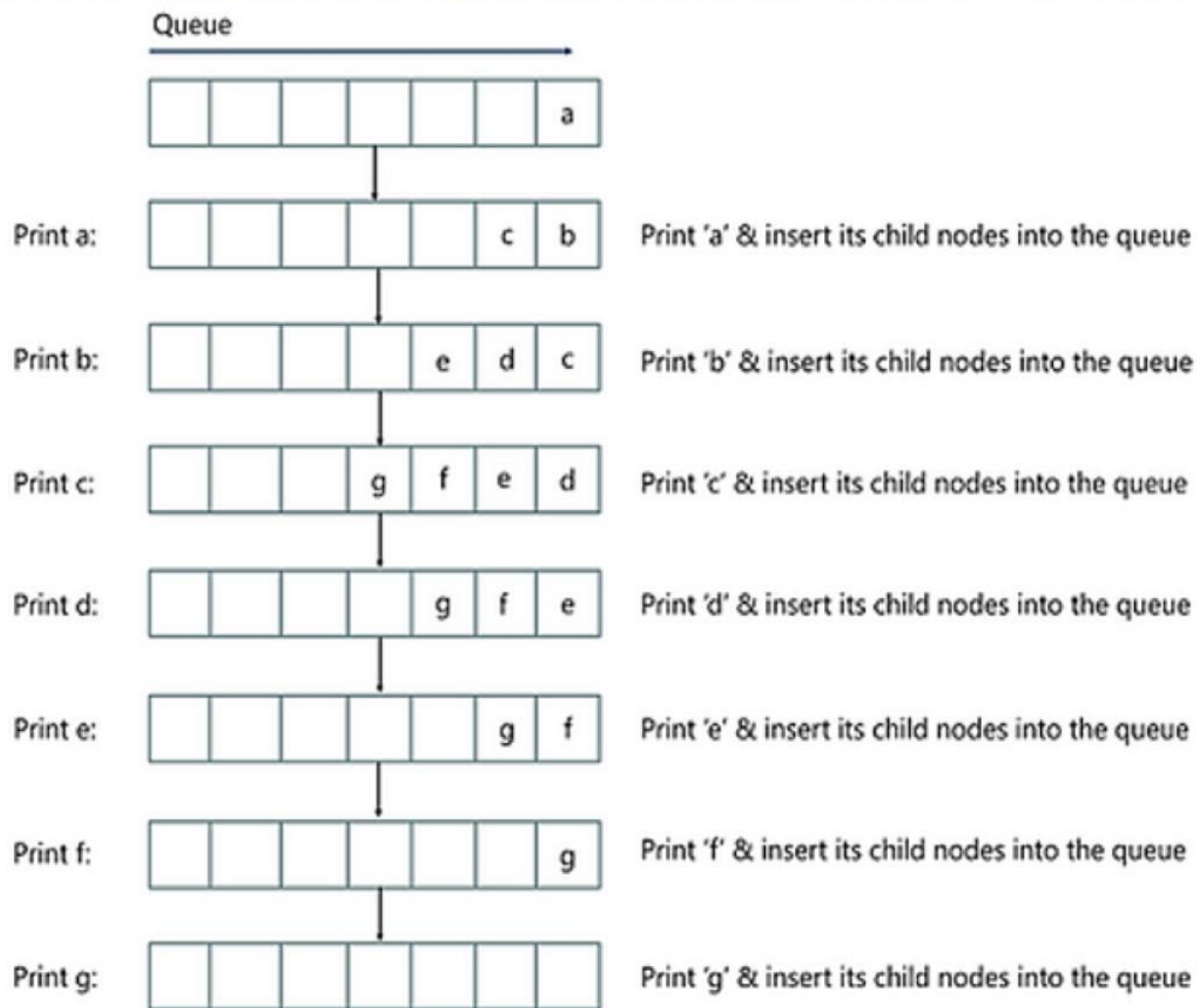
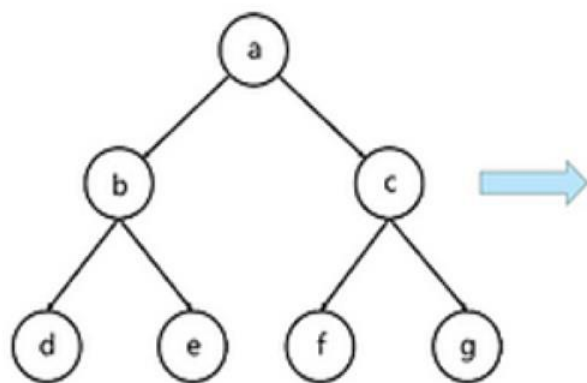
✓ Traversal: Breadth First Search Algorithm (Level-Order)

Dr. Mahmoud Alnamoly

```
void BFS(Node* root) {  
    if (root == nullptr) return;  
    queue<Node*> q;  
    q.push(root);  
  
    while (!q.empty()) {  
        Node* node = q.front();  
        q.pop();  
        cout << node->data << " ";  
        if (node->left != nullptr) q.push(node->left);  
        if (node->right != nullptr) q.push(node->right);  
    }  
}
```

Dr. Mahmoud Alnamoly





Binary Tree: Operations

Dr. Mahmoud Alnamoly

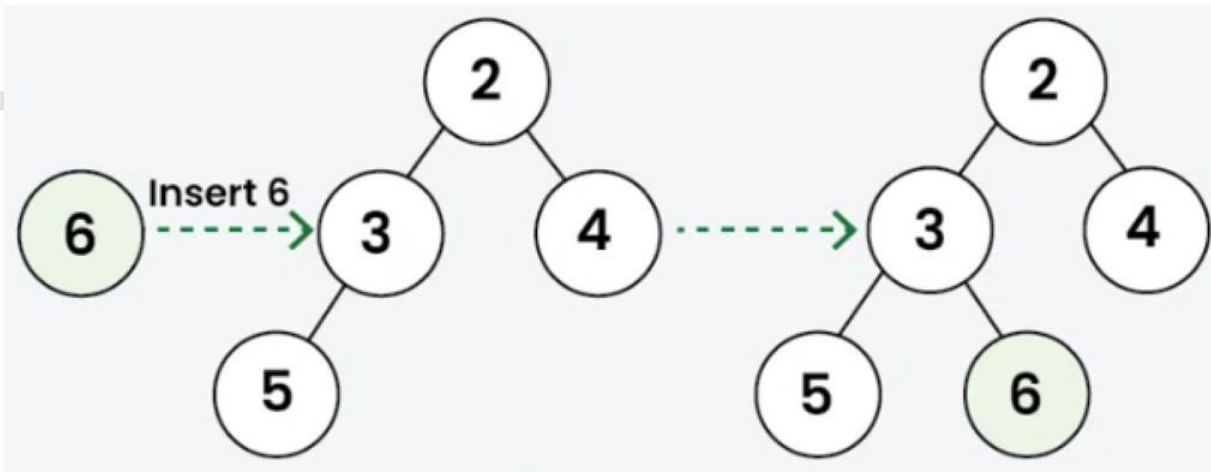
✓ 2. Insertion

As we know that there is no such ordering of elements in the binary tree, So we do not have to worry about the ordering of node in the binary tree.

We would first creates a root node in case of empty tree. Then subsequent insertions involve iteratively searching for an empty place at each level of the tree.

When an empty left or right child is found then new node is inserted there.

By convention, insertion always starts with the left child node.



Dr. Mahmoud Alnamoly

```
// Function to insert a new node in the binary tree
```

```
Node* insert(Node* root, int key) {  
    // If the tree is empty, create the root node  
    if (root == nullptr) {  
        root = new Node(key);  
        return root;  
    }  
    // Create a queue for level order traversal  
    queue<Node*> q;  
    q.push(root);
```

```

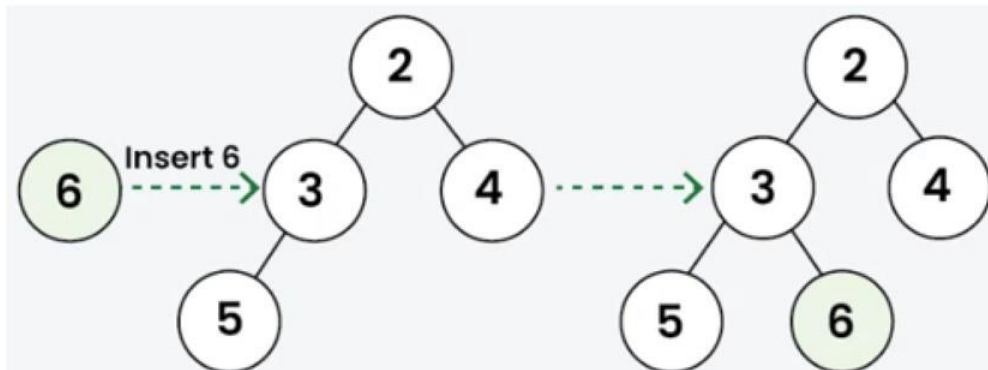
// Do level order traversal until we find an empty place
while (!q.empty()) {
    Node* temp = q.front();
    q.pop();

    // If left child is empty, insert the new node here
    if (temp->left == nullptr) {
        temp->left = new Node(key);
        break;
    }
    else {
        q.push(temp->left);
    }

    // If right child is empty, insert the new node here
    if (temp->right == nullptr) {
        temp->right = new Node(key);
        break;
    }
    else {
        q.push(temp->right);
    }
}
return root;
}

```

Dr. Mahmoud Alnamoly



Binary Tree: Operations

Dr. Mahmoud Alnamoly

✓ 3. Searching

We typically use any traversal method to search. The most common methods are depth-first search (DFS) and breadth-first search (BFS).

In DFS, we start from the root and explore the depth nodes first.

In BFS, we explore all the nodes at the present depth level before moving on to the nodes at the next level.

We continue this process until we either find the node with the desired value or reach the end of the tree. If the tree is empty or the value isn't found after exploring all possibilities, we conclude that the value does not exist in the tree.


```
// Function to search for a value in the binary tree using DFS
bool searchDFS(Node* root, int value) {
    // Base case: If the tree is empty or we've reached a leaf node
    if (root == nullptr) return false;

    // If the node's data is equal to the value we are searching for
    if (root->data == value) return true;

    // Recursively search in the left and right subtrees
    bool left_res = searchDFS(root->left, value);
    bool right_res = searchDFS(root->right, value);

    return left_res || right_res;
}
```

```
int value = 6;
if (searchDFS(root, value))
    cout << value << " is found in the binary tree" << endl;
else
    cout << value << " is not found in the binary tree" <<
    endl;
```


Operation	Time Complexity	Auxiliary Space
In-Order Traversal	$O(n)$	$O(n)$
Pre-Order Traversal	$O(n)$	$O(n)$
Post-Order Traversal	$O(n)$	$O(n)$
Insertion (Unbalanced)	$O(n)$	$O(n)$
Searching (Unbalanced)	$O(n)$	$O(n)$
Deletion (Unbalanced)	$O(n)$	$O(n)$

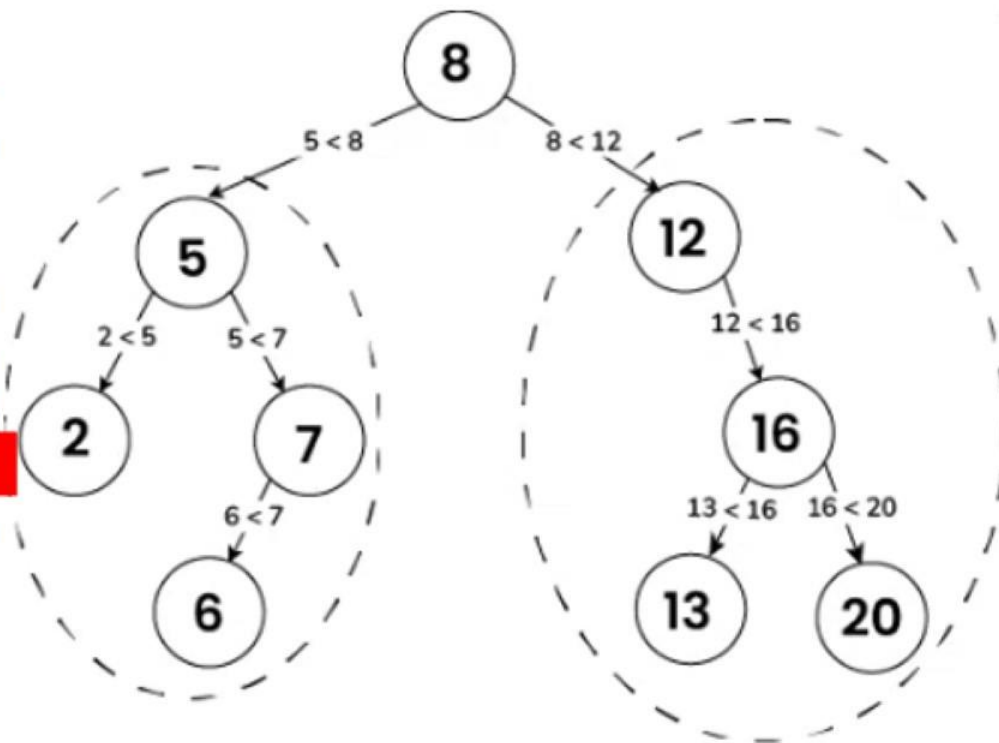
Binary Search Tree

BST: Definition

It possess all the properties of Binary Trees including some extra properties

The values in the left subtree are always less than the values in the root node and the values in the right subtree are always greater than the values in the root node

This hierarchical structure allows for efficient Searching, Insertion, & Deletion operations on the data stored in the tree.



Left subtree contains all elements less than 8

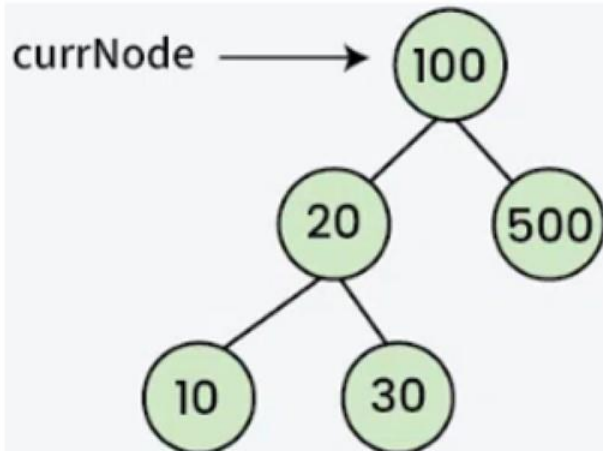
Right subtree contains all elements greater than 8

left subtree < root node ≤ right

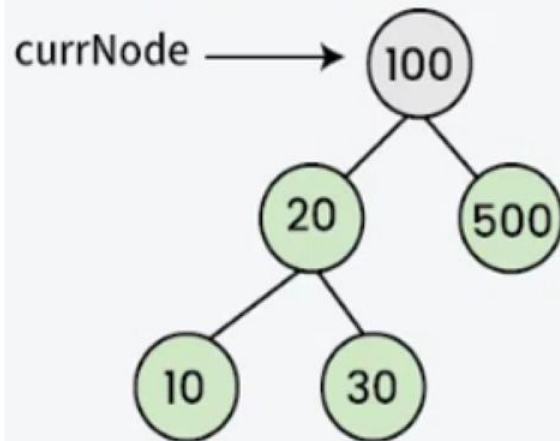
BST: Operations

Dr. Mahmoud Alnamoly

✓ 1. Insertion



key = 40 (the node to be inserted)

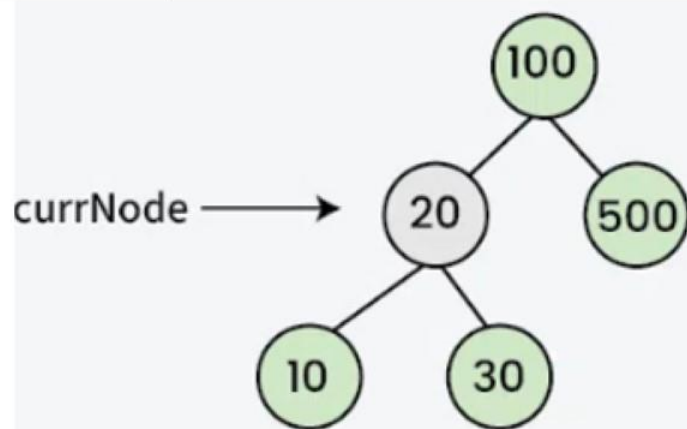


Since 100 is greater than key(40), move the pointer to the left child (20).

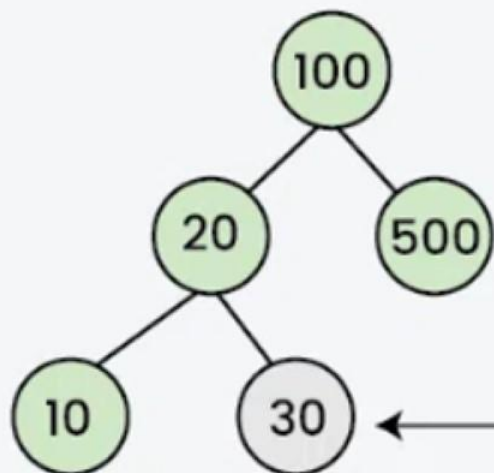
BST: Operations

Dr. Mahmoud Alnamoly

✓ 1. Insertion



Since 20 is less than key(40), move the pointer to the right child (30).

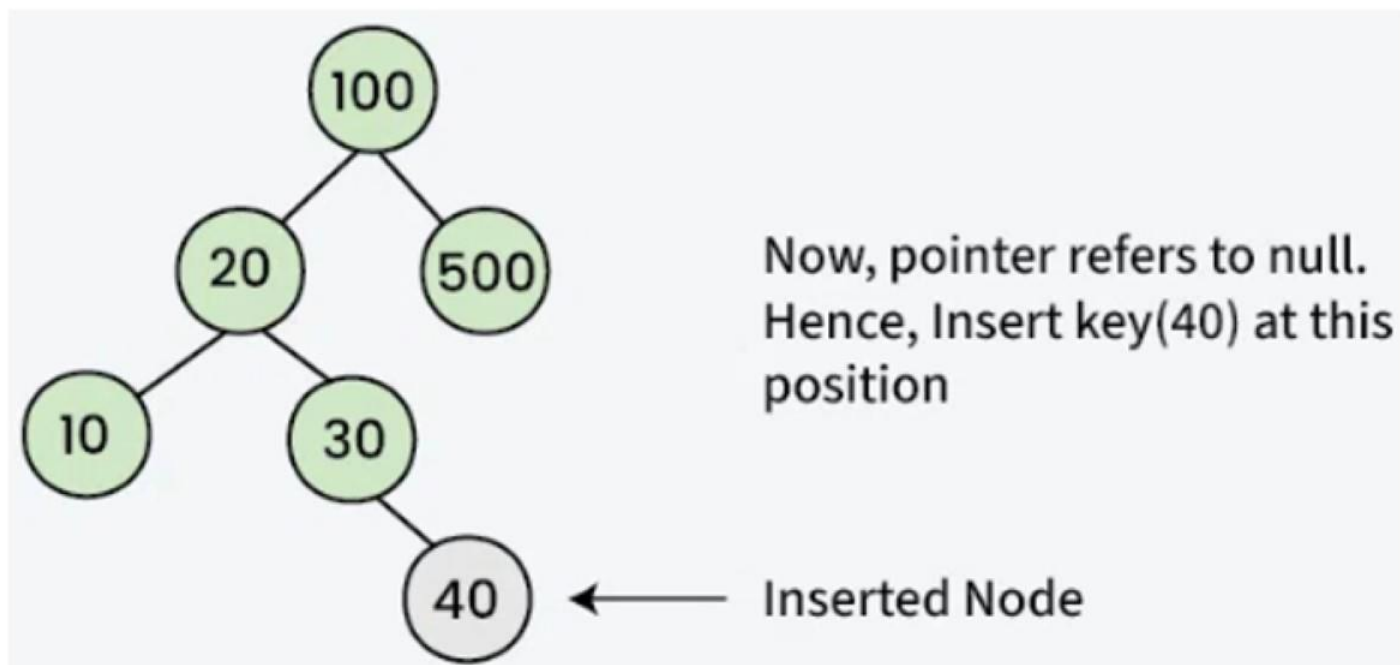


Again, 40 is greater than key(30), move the pointer to the right of 30.

BST: Operations

Dr. Mahmoud Alnamoly

✓ 1. Insertion



BST: Operations

Dr. Mahmoud Alnamoly

✓ 1. Insertion

```
Node* insert(Node* node, int key) {  
    // If the tree is empty, return a new node  
    if (node == NULL)  
        return new Node(key);  
  
    // If the key is already present in the tree, return the node  
    if (node->key == key)  
        return node;  
  
    // Otherwise, recur down the tree, If the key to be inserted is greater than  
    the node's key, insert it in the right subtree  
    if (node->key < key)  
        node->right = insert(node->right, key);  
  
    // If the key to be inserted is smaller than the node's key, insert it in the  
    left subtree  
    else  
        node->left = insert(node->left, key);  
    return node; // Return the (unchanged) node pointer  
}
```

Dr. Mahmoud Alnamoly

BST: Operations

Dr. Mahmoud Alnamoly

✓ 1. Insertion

```
// Driver program to test the above functions
int main() {
    // Creating the following BST
    //      50
    //     /  \
    //    30   70
    //   / \  / \
    //  20 40 60 80

    Node* root = new Node(50);
    root = insert(root, 30);
    root = insert(root, 20);
    root = insert(root, 40);
    root = insert(root, 70);
    root = insert(root, 60);
    root = insert(root, 80);

    // Print inorder traversal of the BST
    inorder(root);

    return 0;
}
```

BST: Operations

Dr. Mahmoud Alnamoly

✓ 2. Search

```
// function to search a key in a BST
```

```
Node* search(Node* root, int key) {
```

```
    // Base Cases: root is null or key
```

```
    // is present at root
```

```
    if (root == NULL || root->key == key) {  
        return root;  
    }
```

```
    // Key is greater than root's key
```

```
    if (root->key < key)  
        return search(root->right, key);
```

```
    // Key is smaller than root's key
```

```
    return search(root->left, key);
```

```
}
```

```
int main() {
```

```
    Node* root = new Node(50);
```

```
    root->left = new Node(30);
```

```
    root->right = new Node(70);
```

```
    root->left->left = new Node(20);
```

```
    root->left->right = new Node(40);
```

```
    root->right->left = new Node(60);
```

```
    root->right->right = new Node(80);
```

```
    (search(root, 19) != NULL) ? cout <<  
    "Found\n" :
```

```
        cout << "Not Found\n";
```

```
    (search(root, 80) != NULL) ? cout <<  
    "Found\n" :
```

```
        cout << "Not Found\n";
```

```
}
```


Advantages of BST over other data structures

Binary Search Trees are more efficient than Binary Trees since time complexity for performing various operations reduces.

Since the order of keys is based on just the parent node, searching operation becomes simpler.

Disadvantages of BST over other data structures

One disadvantage of Binary Search Trees is that if all elements in nodes are either greater than or lesser than the root node, the tree becomes skewed. Simply put, the tree becomes slanted to one side completely.

This skewness will make the tree a linked list rather than a BST, since the worst case time complexity for searching operation becomes $O(n)$.

To overcome this issue of skewness in the Binary Search Trees, the concept of Balanced Binary Search Trees was introduced.

Graphs

>>> Next Lecture